

Model Training Guide — Churn Prediction

Service: prediction-service **Training script:** `scripts/train_models.py` **Model output:** `models/champion/churn/xgboost_churn_v1.json` **Registry:** `models/registry.json` **Last updated:** 2026-08-19

1. Quick Start

```
Set-Location "c:\path\to\customer-lifecycle-ai"  
.\.venv\Scripts\python.exe scripts\train_models.py
```

What happens:

1. Loads features from `customer_features` (training dates)
2. Excludes leakage + dead features (including `id`, `computed_at` metadata)
3. Trains XGBoost with class weighting
4. Evaluates on a holdout date
5. Fits a probability calibrator (Platt + isotonic) out-of-fold
6. Saves the model + calibrator + generates plots + writes health/drift/hallucination metrics
7. Updates `models/registry.json`

2. Configuration — Where It Lives & What It Does

2.1 Shared config (`shared/config/settings.py`)

All training config is env-overridable via `.env` (pydantic-settings).

Env var	Default	What it does
<code>TRAINING_DATES</code>	<code>2026-07-17,2026-07-22</code>	Comma-separated <code>as_of_date</code> values used for training
<code>TRAINING_HOLDOUT_DATE</code>	<code>2026-07-27</code>	Holdout <code>as_of_date</code> for evaluation
<code>LABEL_CHURN_COLUMN</code>	<code>rel_customer_status</code>	Column that defines churn
<code>LABEL_CHURN_POSITIVE_VALUE</code>	<code>Closed</code>	Value meaning "churned" (→ label 1)
<code>TABLE_CUSTOMER_FEATURES</code>	<code>customer_features</code>	Feature table name
<code>COL_CUSTOMER_ID</code> / <code>COL_AS_OF_DATE</code>	<code>customer_id</code> / <code>as_of_date</code>	Identifier columns (excluded from training)

These let you retrain on real ABSA data **without code changes** — point the table/column names at the real schema and set new training dates.

2.2 Leakage guard (`scripts/train_models.py`, `_get_leakage_features`)

Features that encode recency/target information are auto-excluded from training to prevent data leakage. The label column is added dynamically.

2.3 Dead-feature scan (automatic)

Before training, the script scans all numeric columns and auto-excludes any that are `ALL_ZERO` or 100% NULL — these carry no signal.

2.4 Hyperparameters (edit in `train_churn_model()`, `XGBClassifier(...)`)

Parameter	Default	Purpose
<code>max_depth</code>	4	Tree depth — lower = less overfit
<code>learning_rate</code>	0.05	Step size
<code>n_estimators</code>	50	Number of trees
<code>min_child_weight</code>	2	Min leaf weight — higher = more regularized
<code>reg_lambda</code>	1.0	L2 regularization on leaf weights
<code>subsample</code>	0.8	Row sampling per tree
<code>colsample_bytree</code>	0.8	Column sampling per tree (reduces overfit)
<code>scale_pos_weight</code>	auto	Class-imbalance balancing

3. Outputs

3.1 Model artifacts

Path	Content
<code>models/champion/churn/xgboost_churn_v1.json</code>	Trained XGBoost model
<code>models/champion/churn/churn_calibrator_isotonic.joblib</code>	Fitted probability calibrator
<code>models/registry.json</code>	Model metadata, metrics, calibration, training features

3.2 Plots (saved to `models/champion/churn/plots/`)

File	What it shows
<code>roc_curve.png</code>	ROC curve with AUC
<code>calibration.png</code>	Reliability diagram (raw predicted vs actual)
<code>calibration_comparison.png</code>	Raw vs Platt vs Isotonic reliability curves
<code>feature_importance.png</code>	Top-10 feature importances

3.3 Metrics written to `registry.json`

```
"metrics": {  
  "auc": 0.8117,  
  "brier": 0.2115,  
  "ece": 0.2724,  
  "log_loss": 0.6128  
}
```

The `calibrator` field carries the post-calibration comparison:

```
"calibrator": {  
  "method": "isotonic",  
  "ece_raw": 0.4038,  
  "ece_platt": 0.0228,  
  "ece_isotonic": 0.0172,  
  "ece_after": 0.0172,  
  "brier_after": ...,  
  "log_loss_after": ...  
}
```

4. Health, Drift & Hallucination Diagnostics

The training script now computes three diagnostic groups, stored under `diagnostics` in `registry.json`:

4.1 Health (`diagnostics.health`)

Field	Meaning	Red flag
<code>model_health.overfit_gap</code>	train AUC – test AUC	> 0.03 = overfitting
<code>model_health.overfit_flag</code>	bool	<code>true</code> → reduce <code>max_depth/n_estimators</code>
<code>data_health.test_missing_ratio</code>	fraction of NaN in holdout	> 0.05 = data problem
<code>data_health.test_samples</code>	holdout size	too small → unreliable metrics

4.2 Drift (`diagnostics.drift`)

Population Stability Index (PSI) between training and holdout feature distributions. Interpretation:

PSI	Level	Action
< 0.10	low	OK
0.10 – 0.25	moderate	Investigate feature drift

PSI	Level	Action
> 0.25	high	Retrain / review features

`top_drifted_features` lists the 10 most-drifted features.

4.3 Hallucination (`diagnostics.hallucination`)

`overconfidence_ratio` = fraction of high-confidence (≥ 0.9) predictions that are wrong. This proxies "model hallucination" — being confidently incorrect.

Ratio	Meaning
0.00 – 0.10	healthy
0.10 – 0.30	needs calibration
> 0.30	model is overconfident — investigate

5. Probability Calibration

The XGBoost model has strong ranking (AUC 0.81) but raw probabilities are uncalibrated (weighted ECE ~0.40). The training script now fits a calibrator and the prediction service applies it at inference time.

5.1 How it works (leakage-free)

1. Train the base XGBoost model on training dates
2. Generate **out-of-fold** predictions on the training set via `StratifiedKFold(5) + cross_val_predict` (the calibrator fitting data)
3. Fit two calibrators:
 - **Platt scaling** — `LogisticRegression(C=999999)` on OOF probabilities
 - **Isotonic regression** — `IsotonicRegression(out_of_bounds="clip")`
4. Apply both to the holdout set, compute ECE with the weighted formula
5. Save the calibrator with the **lowest holdout ECE**

5.2 Method selection

	Platt	Isotonic
Parametric	Monotonic sigmoid	Non-parametric step function
Dataset size	Small (<1k)	Large (>1k)
Overfit risk	Low	Higher on sparse data

Our calibration set (~10k OOF samples) favours isotonic.

5.3 Results (current model)

Method	ECE
Uncalibrated	0.4038

Method	ECE
Platt	0.0228
Isotonic 	0.0172

5.4 Inference wiring

The prediction service (`app/models/churn_predictor.py`) loads the calibrator from the registry and applies it via `_apply_calibrator()`, which handles both `LogisticRegression` (`predict_proba`) and `IsotonicRegression` (`predict`). `churn_probability` is now a calibrated probability, not a raw ranking score.

6. How to Tune

Goal	Change
Improve AUC	Increase <code>max_depth</code> (4→5), <code>n_estimators</code> (50→100), but watch overfit gap
Reduce overfitting	Lower <code>max_depth</code> , raise <code>min_child_weight/reg_lambda</code> , reduce <code>colsample_bytree</code>
Fix class imbalance	<code>scale_pos_weight</code> is auto-computed; adjust manually if needed
Fix drift	Revisit training dates — ensure training window represents recent data
Fix overconfidence	Calibration is now automatic — check <code>calibrator.ece_after</code>

Workflow:

1. Run training → check `diagnostics`
2. If `overfit_flag` → reduce complexity → re-run
3. If `drift_level` is high → update `TRAINING_DATES` → re-run
4. Compare `metrics.auc` against the ≥ 0.80 target
5. Verify `calibrator.ece_after` < 0.05 (calibrated)

7. Related Scripts

Script	Purpose
<code>scripts/feature_importance.py</code>	Standalone feature importance report (top-N)
<code>services/decision-intelligence-service/scripts/train_ranking_model.py</code>	LightGBM NBA ranking model (Phase 2)

8. Feature Config Reference

Feature computation is separate from model training. See [docs/ml/feature-catalog.md](#) for the 21 features and their env toggles ([FE_ENABLE_PHASE2_DERIVED](#), [FE_ENABLE_PHASE2B_RATIO](#)).